

Algorithms & Data Structures

Weeks 3-4: Elementary Data Structures and Their Processing

Alexander Bak Thorsen, Viktor Kirkeby Førreisdahl
Bertram Agerskov, Jonas Harwig Mejlholm

September 22, 2026

Øvelse 1

(A) Implementer List ADT med en enkeltkædet liste

Vi har implementeret den abstrakte datatype som en singly linked list. Klassen er en template, så den kan håndtere generiske datatyper. Den kan ses i *Code block 1*.

```
1  template <typename Object>
2  class List {
3  private:
4      struct Node {
5          Object data;
6          Node *next;
7      };
8      int theSize;
9      Node *head;
10     Node *tail;
11
12 public:
13     List() {
14         theSize = 0;
15         head = new Node; tail = new Node;
16         head->next = tail;
17         tail->next = nullptr;
18     }
19
20     ~List() { clear(); delete head; delete tail; }
21
22     int size() { return theSize; }
23     bool empty() { return (size() == 0); }
24
25     void clear();
26     void insert(const Object x, int pos);
27     void push_front(const Object x);
28     void push_back(const Object x);
29     void reverse();
30     void print();
31     Object remove(int pos);
32     Object pop_front();
33     Object pop_back();
34     Object find_kth(int pos);
35     bool contains(Object x);
36 };
```

Code Block 1: List-klassen

Funktion	Implementering
List(), ~List()	Constructor og destructor. List() laver et object med head , tail og størrelsen på listen. Destructoren rydder listen med clear() og sletter derefter head og tail .
int size()	Returnerer antallet af elementer.
bool empty()	Kontrollerer om antallet af elementer er nul. Returnerer <i>bool</i>
void push_front(const Object x)	Indsætter en ny node med data <i>x</i> efter head og inkrementerer theSize .
void push_back(const Object x)	Finder sidste element, indsætter en ny node med data <i>x</i> før tail og inkrementerer theSize .
void insert(const Object x, int pos)	Finder noden før <i>pos</i> , forbinder den nye node med data <i>x</i> med resten af listen og inkrementerer theSize .
Object pop_front()	Popper første elementet fra listen, kobler den nye første node til head , dekrementerer theSize og returnerer værdien af det poppede element.
Object pop_back()	Finder noden før det sidste elementet, kobler sidste element ud, kobler det nye sidste element til tail , dekrementerer theSize og returnerer værdien på det poppede element.
Object remove(int pos)	Finder elementet før <i>pos</i> , popper elementet på index <i>pos</i> , kobler listen sammen igen, dekrementerer theSize og returnerer værdien af det poppede element
Object find_kth(int pos)	Gennemløber listen til <i>pos</i> og returnerer værdien.
bool contains(Object x)	Sammenligner elementerne med <i>x</i> og returnerer true ved et match, ellers false.
void print()	Gennemløber og udskriver elementerne uden at ændre listen.
void clear()	Sletter alle elementnoder, forbinder head med tail og nulstiller theSize .

Table 1: Oversigt over List-funktionernes implementering

Nedenfor i *Table 1* ses en oversigt over de forskellige funktioner til List-klassen og en kort beskrivelse af deres implementering. Ugyldige positioner og fjernelse fra en tom liste håndteres med `std::out_of_range`.

Eksempel: insert()

Den fulde implementering af `insert()` kan ses i *Code Snippet 2*. For resten af implementeringerne henvises til `Exercise_01/simple_list.hpp`.

```
1  template <typename Object>
2  void List<Object>::insert(const Object x, int pos)
3  {
4      if (pos < 0 || pos > theSize)    // Tjekker om pos er gyldig
5      {
6          throw std::out_of_range("Position is out of range");
7      }
8
9      Node* p = head;    // Initialiserer pointer p
10     for (int i = 0; i < pos; i++)    // Løber gennem listen til noden før noden ved index pos
11     {
12         p = p->next;
13     }
14
15     Node* pNew = new Node;    // Opretter ny node
16     pNew->data = x;    // Sætter den nye nodes værdi til x
17     pNew->next = p->next;    // Sætter den nye nodes next pointer til p-nodens next pointer
18     p->next = pNew;    // Sikrer p-nodens next pointer nu peger paa den nye node
19     theSize++;    // Inkrementerer størrelsen paa listen
20 }
```

Code Block 2: Implementering af insert()

(B) Implementer og test reverse af listen

Implementering af `reverse()` kan ses i *Code Snippet 3*. Den næste node gemmes, før forbindelsen vendes. `pre` starter ved `tail`, så det oprindelige første element bliver forbundet med `tail`. Til sidst forbindes `head` med det nye første element.

```
1  template <typename Object>
2  void List<Object>::reverse()
3  {
4      Node* pre = tail;
5      Node* cur = head->next;
6
7      while (cur != tail)
8      {
9          Node* next = cur->next;
10         cur->next = pre;
11         pre = cur;
12         cur = next;
13     }
14     head->next = pre;
15 }
```

Code Block 3: Implementering af reverse()

Funktionerne testes separat med `assert`, som kontrollerer værdier, rækkefølge og størrelse. Testene omfatter tomme lister, enkelte og flere elementer samt ugyldige operationer. Listen testes med både `int` og `std::string`. For indsættelse og fjernelse udskrives desuden det forventede og det faktiske resultat.

Eksempel: test_insertion()

Testen i *Code Block 4* kontrollerer indsættelse i starten, midten og slutningen samt indsættelse i en tom liste. Hvis en betingelse i assert ikke er opfyldt, stopper programmet. De resterende tests findes i *Exercise_01/test_simple_list.cpp*.

```
1 void test_insertion()
2 {
3     List<int> list;
4     list.push_front(10);
5     list.push_front(12);
6     list.push_back(14);
7     list.insert(5, 1);
8     list.insert(3, 0);
9     list.insert(20, list.size());
10
11     std::cout << "Expected: 3, 12, 5, 10, 14, 20,\nActual:   ";
12     list.print();
13     std::cout << std::endl;
14     assert(list.size() == 6);
15     int expected[] = {3, 12, 5, 10, 14, 20};
16     for (int i = 0; i < 6; i++)
17         assert(list.find_kth(i) == expected[i]);
18
19     List<int> other;
20     other.push_back(7);
21     assert(other.find_kth(0) == 7);
22     other.clear();
23     other.insert(8, 0);
24     assert(other.find_kth(0) == 8);
25 }
```

Code Block 4: Test af indsættelse

Øvelse 2

(A) Implementer en stack med et array

Nedenfor i Stack H fil kan vi se implementeringen af vores h-fil, hvor vores konstruktør vil sætte den private variabel `size_` til 100, hvis der ingen parametre er i den konstruktør, som bliver kaldt. Er der en parameter med i kaldet, vil den følge konstruktøren og sætte parameteren til `size_`, som kan ses i Stack CPP fil constructor osv. Modsat definerer vi også vores destruktør til at slette vores allokeret array igen, når vi ikke skal bruge det. Variablen `head_` angiver antallet af elementer og dermed indekset for den næste ledige plads. Funktionerne `size()`, `sizeOfStack()` og `top()` bruges til at kontrollere antal, kapacitet og øverste værdi i testene. Til sidst har vi en pointer til vores array, som vi kommer til at lave i konstruktøren.

```
1  #ifndef STACK_H
2  #define STACK_H
3
4  class stack
5  {
6  public:
7      stack(int size = 100); //constructor
8      ~stack(); //destructor
9
10     void pop();
11     void push(int x);
12     int size() const { return head_; }
13     int sizeOfStack() const { return size_; }
14     int top() const;
15
16 private:
17     int size_;
18     int head_;
19     int *array_;
20 };
21
22 #endif
```

Code Block 5: Stack H fil

(B) Fordobl kapaciteten, når stacken er fuld

Nedenfor i Stack CPP fil constructor osv kan vi se implementeringen af stack.cpp hvor vi i konstruktøren sætter size til size_ og derefter head_ til 0, da vi ikke har indsat nogen elementer endnu. Derefter allokerer vi ny plads på heapen til vores nye array med størrelsen **size_** ved at bruge *new*.

Herefter definerer vi vores destruktør, som har til opgave at slette vores allokeret memory igen, når vi er færdige med at bruge det. Destruktøren bruger **delete[]** til at frigive arrayet, når stack-objektet destrueres. I **push()** frigives det gamle array direkte med **delete[]**; dette kalder ikke stackens destruktør.

```
1 #include "stack.h"
2 #include <stdexcept>
3
4 stack::stack(int size) : size_(size), head_(0)
5 {
6     if (size_ <= 0)
7         throw std::invalid_argument("Capacity must be positive");
8     array_ = new int[size_]{};
9 }
10
11 stack::~stack()
12 {
13     delete[] array_;
14 }
```

Code Block 6: Stack CPP fil constructor osv.

Hvis arrayet er fuldt, allokerer **push()** et nyt array med dobbelt kapacitet, kopierer elementerne og frigiver det gamle array. Derefter indsættes den nye værdi præcis en gang, og **head_** øges med en. Konstruktøren afviser kapaciteter på nul eller derunder med **std::invalid_argument**.

```
1 void stack::push(int x)
2 {
3     if (head_ == size_)
4     {
5         int *new_array_ = new int[size_ * 2];
6         for (auto i = 0; i < size_; i++)
7         {
8             new_array_[i] = array_[i];
9         }
10        size_ *= 2;
11        delete[] array_;
12        array_ = new_array_;
13    }
14    array_[head_++] = x;
15 }
```

Code Block 7: Stack CPP fil PUSH metode

`pop()` kontrollerer først, at stacken ikke er tom. Derefter mindskes `head_` med en, så det tidligere øverste element ikke længere tilhører stacken. Det er ikke nødvendigt at nulstille arraypladsen.

```
1 void stack::pop()
2 {
3     if (head_ == 0)
4         throw std::out_of_range("Cannot pop from an empty stack");
5     head_--;
6 }
7
8 int stack::top() const
9 {
10     if (head_ == 0)
11         throw std::out_of_range("Stack is empty");
12     return array_[head_ - 1];
13 }
```

Code Block 8: Stack CPP fil POP metode

(C) Analyser runtime for push og pop

`push()` tager $O(1)$ tid uden udvidelse og $O(N)$ ved udvidelse, hvor N er antallet af gemte elementer. Den amortiserede tid er $O(1)$: ved fordobling danner antallet af kopieringer en geometrisk række, eksempelvis $1 + 2 + 4 + \dots < 2N$, så N indsættelser samlet tager $O(N)$ tid. `pop()` tager $O(1)$ tid, da arbejdet ikke afhænger af antallet af elementer.

Øvelse 3

(A) Implementer en generisk kø med stacks

Vi anvender `std::stack` til at implementere køen. Implementeringen bruger kun stackens offentlige operationer `push()`, `pop()`, `top()` og `empty()` og tilgår hverken dens interne struktur eller iteratorer.

```
1  #ifndef QUEUE_H
2  #define QUEUE_H
3  #include <stack>
4  #include <stdexcept>
5
6  template <typename T>
7  class queue
8  {
9  private:
10     std::stack<T> s1; // To stacks oprettes for at kunne omdanne LIFO til FIFO
11     std::stack<T> s2;
12
13 public:
14     void pushQueue(T x)
15     {
16         s1.push(x); // Vaerdier bliver tilfoejet til en stack
17     }
18
19     /*
20     While loekke laves hvor indlaeste vaerdier rykkes fra den foerste stack til den anden stack
21     for at vende raekkefoelgen om og vaerdierne kan aflaeses/fjernes som FIFO i stedet for LIFO
22     */
23     void popQueue()
24     {
25         while (!s1.empty()) // Vender stacken om
26         {
27             T x = s1.top();
28             s1.pop();
29             s2.push(x);
30         }
31         if (s2.empty()) // tjek om stack er tom foer der fjernes noget fra stackken
32         {
33             throw std::runtime_error("Queue is empty");
34         }
35         s2.pop(); // Fjerne foerste ting fra stacken
36
37         while (!s2.empty()) // Vender stacken om
38         {
39             T x = s2.top();
40             s2.pop();
41             s1.push(x);
42         }
43     }
44
45     T firstInQueue() // Laves af samme grund som popQueue
46     {
47         T first; // Instalisere en standard vaerdi til at indehold foerste vaerdi
48         while (!s1.empty())
49         {
50             T x = s1.top();
51             s1.pop();
52             s2.push(x);
53         }
54         if (s2.empty())
55         {
56             throw std::runtime_error("Queue is empty");
57         }
58         first = s2.top();
59     }
```

```

60     while (!s2.empty())
61     {
62         T x = s2.top();
63         s2.pop();
64         s1.push(x);
65     }
66     return first; // Retunere den foerste tilfoejet vaerdi til stacken
67 }
68 };
69
70 #endif

```

Code Block 9: Omdannelse af stack til Queue

Øvelse 4

Vi indsætter keysne 5, 28, 19, 15, 20, 33, 12, 17, 33, 10 i en hashtabel med størrelse 7 og hashfunktion $h(x) = x \bmod 7$. For hver metode bestemmes tabellens indhold og loadfaktor.

Når vi skal indsætte noget i hashtabellen, bruges følgende ligning:

$$start(x) = H(x) \bmod M \quad (1)$$

Hvor vi i vores tilfælde har en $H(x)$, som hedder:

$$H(x) = x \bmod 7$$

Hvis vi bruger chaining, betyder det ifølge powerpointen, at "chaining stores colliding keys in the same bucket", og vi derved blot tilføjer keysne efter hinanden, hvis de ender på samme indeks.

Hvis vi bruger linear probing eller quadratic probing, kan man i stedet bruge følgende formel til at finde frem til det indeks, som vi skal indsætte på, hvis det indeks, som man finder i startfunktionen, allerede er fyldt. På den måde kan man ikke gemme flere værdier i samme bucket:

$$h_i(x) = (start(x) + f(i)) \bmod M \quad (2)$$

Her er $i = 0, 1, 2, \dots$ forsøgsnummeret, mens $f(i)$ afhænger af metoden. Vi behandler gentagne keys som separate indsættelser; derfor tælles begge forekomster af 33 ved chaining.

(A) Indsæt keys med chaining og beregn loadfaktoren

Ved chaining bruges kun startfunktionen(1), og der vises et eksempel på, hvordan vi kan indsætte elementer. Vi starter med 5:

$$start(5) = H(5) \bmod 7 = 5$$

5 indsættes på indeks 5:

Value						5	
Index	0	1	2	3	4	5	6

Figure 1: Indsætter 5

Derefter fortsætter vi med alle keys. Hvis vi lander på samme indeks indsætter vi bare tallet efter det allerede indsatte og derved ender vi med følgende hash map:

Value	28	15		17, 10		5, 19, 33, 12, 33	20
Index	0	1	2	3	4	5	6

Figure 2: Hashmap ved brug af chaining

Load factor

Formlen for loadfaktoren ser ud som følgende:

$$\lambda = \frac{n}{M}, \quad n = \text{entries}, \quad M = \text{capacity} \quad (3)$$

Derved ender vores hashtabel med en loadfaktor på:

$$\lambda = \frac{10}{7} \approx 1.43$$

Vi kan se på Hashmap ved brug af chaining, at vores data er vildt dårligt spredt ud, og det påvirker vores performance negativt, selvom vores loadfaktor faktisk ikke er særlig stor. Vores testdata er nok valgt for at fremvise hvordan chaining kan skabe noget clustering og fremvise den dårlige side af chaining.

Da chaining gør det muligt at have flere keys i hvert bucket, kan vores loadfaktor blive over 1 i modsætning til de næste to metoder.

(B) Indsæt keys med linear probing og beregn loadfaktoren

Ved linear probing er

$$f(i) = i$$

som bruges i formel 2, hvis det indeks, som formel 1 returnerer, allerede indeholder en key.

Vi starter igen med at indsætte 5, som placeres på indeks 5(1). Derefter indsætter vi 28, som ender på indeks 0(1). Når vi så bagefter prøver at indsætte 19 får vi indeks 5 retur(1), men den indeholder allerede en key. Derfor kan vi nu bruge vores nye formel(2) til at finde en ny plads. Vi inkrementere i med 1 og prøver igen:

$$h_i(19) = (\text{start}(19) + f(1)) \bmod 7 = (5 + 1) = 6$$

Derved kan vi placere key 19 på indeks 6, da denne var fri. **Hvis** indeks 6 var optaget i forvejen, ville vi blot inkrementere i med 1 igen, og så ville regnestykket se sådan ud:

$$h_i(19) = (\text{start}(19) + f(2)) \bmod 7 = (5 + 2) = 7 \bmod 7 = 0$$

Hvis indeks 0 også er optaget, gør vi det samme igen, indtil vi finder en ledig plads. Dette gøres med alle vores entries og derved kommer vores hashtabel til at se sådan ud:

Value	28	15	20	33	12	5	19
Index	0	1	2	3	4	5	6

Figure 3: Hashmap ved brug af linear probing

Som vi kan se på Hashmap ved brug af linear probing er det ikke alle vores keys, som er kommet med i hashmappet, da linear probing ikke tillader, at der kommer flere keys i hver bucket. Vi har kunnet tilføje de 7 første (5, 28, 19, 15, 20, 33 og 12), men de 3 sidste (17, 33 og 10) kan ikke tilføjes.

Load factor

Vi bruger formel 3 til at finde load faktoren ved linear probing:

$$\lambda = \frac{7}{7} = 1$$

Hvilket giver god mening, da den er fyldt.

(C) Indsæt keys med quadratic probing og beregn loadfaktoren

Ved quadratic probing er

$$f(i) = i^2$$

som bruges i formel 2, hvis det indeks, som formel 1 returnerer, allerede indeholder en key.

Vi starter igen med at indsætte 5, som placeres på indeks 5(1). Derefter indsætter vi 28, som ender på indeks 0(1). Når vi så bagefter prøver at indsætte 19, får vi indeks 5 retur(1), men den indeholder allerede en key. Derfor kan vi nu bruge vores nye formel(2) til at finde en ny plads. Vi inkrementere i med 1 og prøver igen:

$$h_i(19) = (start(19) + f(1)) \bmod 7 = (5 + 1^2) = 6$$

Derved kan vi placere key 19 på indeks 6, da denne var fri. **Hvis** indeks 6 var optaget i forvejen, ville vi blot inkrementere i med 1 igen, og så ville regnestykket se sådan ud:

$$h_i(19) = (start(19) + f(2)) \bmod 7 = (5 + 2^2) = 9 \bmod 7 = 2$$

Hvis indeks 2 også er optaget, gør vi det samme igen, indtil vi finder en ledig plads. Dog kan vi ende ud i, at vores key aldrig kan komme ind i vores hashmap, da størrelsen på vores hashmap er et printal på 7, hvilket betyder, at vi maksimalt kan tilgå 4 forskellige indeks, når vi inkrementerer i. Det problem rammer vi, når vi forsøger at indsætte vores 12-tal efter 33. De 4 forskellige indeks, som vi kan tilgå, bliver 5, 6, 0, 2, men disse er allerede fyldt, når vi itererer igennem dem. Derfor vil 12 aldrig blive sat ind.

Efter det mislykkede forsøg med 12 indsættes 17: startindekset 3 er optaget, men $(3 + 1^2) \bmod 7 = 4$ er ledigt. Dermed bliver tabellen fuld, og vi kan derved ikke indsætte de sidste to tal (33 og 10), men dette kunne stadig have været tilfældet, hvis de 4 forskellige indeks, som 33 og 10 tilgår, ikke var blandt de ledige, som var tilbage.

Value	28	15	33	20	17	5	19
Index	0	1	2	3	4	5	6

Figure 4: Hashmap ved brug af quadratic probing

Load factor

Vi bruger formel 3 til at finde load faktoren ved quadratic probing:

$$\lambda = \frac{7}{7} = 1$$

Alle pladser ender med at være fyldt, hvilket medfører en loadfaktor på 1 ligesom ved linear probing. Dog er det af en anderledes grund. Linear probing fylder hash-mappet op indekxsvist lige så stille, hvorimod quadratic nogle gange får keys, som ikke kan komme ind og derved skaber ledige pladser undervejs, som vi tilfældigvis rammer med keys, som kommer bagefter.

Øvelse 5

(A) Implementer et Set med en Stack ADT

Der er implementeret et Set ved hjælp af `std::stack`. Der bruges kun stackens offentlige operationer og ingen iteratoren. Implementeringen er beskrevet i kodekommentarerne.

```
1  #ifndef SET_H
2  #define SET_H
3  #include <stack>
4
5  template <typename T>
6  class set
7  {
8  private:
9      std::stack<T> data; // Stack der bruges til at gemme alle værdierne i vores Set
10     std::stack<T> temp; // Midlertidig stack der bruges naar vi gennemløber data og bagefter skal gendanne
        rækkefølgen
11
12     /* Gennemløber hele stacken for at undersøge om en bestemt værdi allerede findes.
13     Værdierne flyttes midlertidigt over i temp, saa data kan gendannes bagefter */
14     bool foundItem(const T &x)
15     {
16         bool found = false;
17         while (!data.empty())
18         {
19             if (x == data.top()) // Hvis værdien findes sættes found til true
20             {
21                 found = true;
22             }
23             temp.push(data.top()); // Flytter værdien midlertidigt væk saa naeste element kan undersøges
24             data.pop();
25         }
26         // Gendanner data efter hele stacken er blevet gennemløbet
27         while (!temp.empty())
28         {
29             data.push(temp.top());
30             temp.pop();
31         }
32         return found;
33     }
34
35 public:
36     /* Tilføjer kun værdien hvis den ikke allerede findes,
37     da et Set ikke maa indeholde duplicates */
38     void addItem(const T &x)
39     {
40         if (!foundItem(x))
41         {
42             data.push(x);
43         }
44     }
45
46     /* Gennemløber hele Settet og flytter alle værdier undtagen den der skal fjernes
47     over i temp, Derefter gendannes data uden den fjernede værdi. */
48     void removeItem(const T &x)
49     {
50         while (!data.empty())
51         {
52             if (!(x == data.top())) // Hvis det ikke er værdien der skal fjernes gemmes den midlertidigt
53             {
54                 temp.push(data.top());
55                 data.pop();
56             }
57             else // Hvis værdien matcher fjernes den ved ikke at gemme den i temp
58             {
```

```

59         data.pop();
60     }
61 }
62 while (!temp.empty())
63 {
64     data.push(temp.top());
65     temp.pop();
66 }
67 }
68
69 // Returnerer true hvis vaerdien findes i Set ellers false
70 bool setContains(const T &x)
71 {
72     return foundItem(x);
73 }
74
75 /* Laver et nyt Set der indeholder alle vaerdier fra begge Sets
76 addItem soerger selv for at duplicates ikke bliver tilfoejet */
77 set<T> setUnion(set<T> &other)
78 {
79     set<T> result;
80     std::stack<T> tempUnion; // Midlertidig stack der bruges til at gendanne de oprindelige Sets bagefter
81     while (!data.empty()) // Tilfoejer alle vaerdier fra det foerste Set til resultatet
82     {
83         result.addItem(data.top());
84         tempUnion.push(data.top());
85         data.pop();
86     }
87     while (!tempUnion.empty()) // Gendanner det foerste Set
88     {
89         data.push(tempUnion.top());
90         tempUnion.pop();
91     }
92     while (!other.data.empty()) // Tilfoejer alle vaerdier fra det andet Set til resultatet
93     {
94         result.addItem(other.data.top());
95         tempUnion.push(other.data.top());
96         other.data.pop();
97     }
98     while (!tempUnion.empty()) // Gendanner det andet Set
99     {
100         other.data.push(tempUnion.top());
101         tempUnion.pop();
102     }
103     return result;
104 }
105
106 // Laver et nyt Set med de vaerdier som findes i begge Sets
107 set<T> setIntersection(set<T> &other)
108 {
109     set<T> result;
110     std::stack<T> tempInter; // Midlertidig stack der bruges til at gendanne det foerste Set bagefter
111
112     while (!data.empty())
113     {
114         T value = data.top();
115         if (other.setContains(value)) // Tilfoejer kun vaerdien hvis den ogsaa findes i det andet Set
116         {
117             result.addItem(data.top());
118         }
119         tempInter.push(data.top());
120         data.pop();
121     }
122     while (!tempInter.empty())
123     {
124         data.push(tempInter.top());

```

```

125         tempInter.pop();
126     }
127     return result;
128 }
129
130 /* Laver et nyt Set med de vaerdier som findes i det foerste Set,
131 men ikke findes i det andet Set */
132 set<T> setDifference(set<T> &other)
133 {
134     set<T> result;
135     std::stack<T> tempDiff;
136
137     while (!data.empty())
138     {
139         T value = data.top();
140         if (!other.setContains(value)) // Tilfoejer kun vaerdien hvis den ikke findes i det andet Set
141         {
142             result.addItem(data.top());
143         }
144         tempDiff.push(data.top());
145         data.pop();
146     }
147     while (!tempDiff.empty())
148     {
149         data.push(tempDiff.top());
150         tempDiff.pop();
151     }
152     return result;
153 }
154 };
155
156 #endif

```

Code Block 10: Omdannelse af stack til Set

Øvelse 6

(A) Implementer en Dictionary med vector og pair

Opgaven består i at implementere en Dictionary/Map ved hjælp af en STL `vector`. Hvert element i vektoren implementeres som et `std::pair`, hvor `.first` = key og `.second` = value.

Følgende key Dictionary-operations blev implementeret: `insert()`, `remove()`, `get()` og `update()`.

```
1  #pragma once
2  #include <vector>
3  #include <utility>
4  #include <iostream>
5
6  class Dictionary
7  {
8  private:
9      std::vector<std::pair<int, int>>> data;
10
11 public:
12     void print() const
13     {
14         for (const auto& entry : data)
15             std::cout << entry.first << ": " << entry.second << '\n';
16     }
17
18     bool insert(int key, int value) // Add a key-value pair to the dictionary
19     {
20         for (const auto& entry : data) // check if key already exists
21         {
22             if (entry.first == key)
23             {
24                 return false;
25             }
26         }
27
28         data.push_back({key, value});
29         return true;
30     }
31
32
33     bool remove(int key) // Remove a key-value and its associated value
34     {
35         for (auto it = data.begin(); it != data.end(); ++it)
36         {
37             if (it->first == key)
38             {
39                 data.erase(it);
40                 return true;
41             }
42         }
43
44         return false;
45     }
46
47
48     bool get(int key, int& value) const // Retrieve the value associated with a given key.
49     {
50         for (const auto& entry : data)
51         {
52             if (entry.first == key)
53             {
54                 value = entry.second;
55                 return true;
56             }
57         }
58         return false;
59     }
60 }
```

```

56     }
57 }
58
59     return false;
60 }
61
62
63 bool update(int key, int newValue) // Change the value associated with a given key
64 {
65     for (auto& entry : data)
66     {
67         if (entry.first == key)
68         {
69             entry.second = newValue;
70             return true;
71         }
72     }
73
74     return false;
75 }
76 };

```

Code Block 11: Implementering af Dictionary

Funktionerne testes i main()

```

1  #include "dictionary.h"
2  #include <cassert>
3
4  int main()
5  {
6      Dictionary dictionary;
7
8      dictionary.insert(10, 100);
9      dictionary.insert(20, 200);
10     dictionary.insert(30, 300);
11
12     std::cout << "Initial dictionary:" << std::endl;
13     dictionary.print();
14
15
16     int value;
17
18     if (dictionary.get(20, value))
19     {
20         std::cout << "\nValue for key 20: "
21                 << value << std::endl;
22     }
23     else
24     {
25         std::cout << "\nKey not found." << std::endl;
26     }
27
28
29     if (dictionary.update(20, 250))
30     {
31         std::cout << "\nKey 20 updated." << std::endl;
32     }
33
34     dictionary.print();
35
36
37     if (dictionary.remove(10))
38     {
39         std::cout << "\nKey 10 removed." << std::endl;

```

```

40     }
41
42     dictionary.print();
43
44     assert(!dictionary.insert(20, 999));
45     assert(dictionary.get(20, value));
46     assert(value == 250);
47     assert(!dictionary.get(10, value));
48     assert(!dictionary.remove(99));
49     assert(!dictionary.update(99, 123));
50     std::cout << "Dictionary tests passed!" << std::endl;
51     return 0;
52 }

```

Code Block 12: Test af key-operationer

Øvelse 7

(A) Verificer den givne hashtabel

Når vi skal indsætte noget i hashtabellen fra opgave 7, bruges følgende ligning[1], som vi også har brugt i opgave 4:

$$start(x) = H(x) \bmod M$$

Hvor vi i vores tilfælde har en $H(x)$, som hedder:

$$H(x) = x \bmod 11$$

Derefter kan vi bruge følgende formel til at finde frem til den indeks, som vi skal indsætte på, hvis det indeks, som start returnerer, allerede er fyldt[2]:

$$h_i(x) = (start(x) + f(i)) \bmod M$$

Og da vi bruger **quadratic probing**, vil $f(i)$ se sådan ud:

$$f(i) = i^2$$

Derefter kan vi nu prøve at gendanne vores tabel ved at indsætte 22, 5, 16 og 27 i rækkefølge:

Indsætter 22:

$$start(22) = H(22) \bmod 11 = 0$$

Og da der ingenting er på indeks 0, kan vi indsætte 22:

Value	22										
Index	0	1	2	3	4	5	6	7	8	9	10

Figure 5: Indsætter 22

Derefter prøver vi at indsætte 5:

$$start(5) = H(5) \bmod 11 = 5$$

Der er heller ikke noget på indeks 5 og derved kan vi indsætte vores femtal:

Value	22					5					
Index	0	1	2	3	4	5	6	7	8	9	10

Figure 6: Indsætter 5

Nu prøver vi at indsætte det næste tal, som er 16:

$$start(16) = H(16) \bmod 11 = 5$$

Nu opstår der et problem, da indeks 5 allerede er fyldt. Derved skal vi bruge vores quadratic probing og inkrementere i med 1:

$$h_i(16) = (start(16) + f(1)) \bmod 11 = (5 + 1^2) = 6$$

Derved finder vi ud af, at vores 16-tal skal stå på indeks nr. 6:

Value	22					5	16				
Index	0	1	2	3	4	5	6	7	8	9	10

Figure 7: Indsætter 16

Nu prøver vi til sidst at sætte tallet 27 ind:

$$start(27) = H(27) \bmod 11 = 5$$

Nu rammer vi indeks 5 igen, og vi kan igen ikke indsætte. Derfor må vi igen prøve quadratic probing og inkrementere i med 1 indtil vi finder en ledig plads:

$$h_i(27) = (start(27) + f(1)) \bmod 11 = (5 + 1^2) = 6$$

Efter 1 omgang rammer vi indeks 6, hvor der allerede står et element. Derfor skal vi inkrementere i med 1 og prøve igen:

$$h_i(27) = (start(27) + f(2)) \bmod 11 = (5 + 2^2) = 9$$

Derved finder vi ud af, at nummeret 27 skal stå på indeks nr. 9:

Value	22					5	16			27	
Index	0	1	2	3	4	5	6	7	8	9	10

Figure 8: Indsætter 27

Derved kan vi verificere, at tabellen er korrekt dannet, som vi får den, og at indekserne 1, 2, 3, 4, 7, 8 og 10 alle er ubrugte indtil videre.

(B) Indsæt 1, 12, alder og studienummer

Derefter indsætter vi tallet 1:

$$start(1) = H(1) \bmod 11 = 1$$

Da der ingenting er på indeks 1, kan vi indsætte vores 1-tal:

Value	22	1				5	16			27	
Index	0	1	2	3	4	5	6	7	8	9	10

Figure 9: Indsæt 1

Derefter prøver vi at indsætte det næste tal, som er 12:

$$start(12) = H(12) \bmod 11 = 1$$

Dog er indeks 1 allerede optaget og vi må derfor rykke vores 12-tal til næste ledige plads:

$$h_i(12) = (start(12) + f(1)) \bmod 11 = (1 + 1^2) = 2$$

Da der er plads på indeks 2 og derfor indsættes 12-tallet der:

Value	22	1	12			5	16			27	
Index	0	1	2	3	4	5	6	7	8	9	10

Figure 10: Indsætter 12

Nu prøver vi at indsætte min alder, som er 23 år :D

$$start(23) = H(23) \bmod 11 = 1$$

Da indeks 1 er optaget, må vi tjekke den næste:

$$h_i(23) = (start(23) + f(1)) \bmod 11 = (1 + 1^2) = 2$$

Indeks 2 er også optaget, så vi må inkrementere i med 1 og prøve igen:

$$h_i(23) = (start(23) + f(2)) \bmod 11 = (1 + 2^2) = 5$$

Indeks 5 er også optaget, så vi må inkrementere i med 1 og prøve igen:

$$h_i(23) = (start(23) + f(3)) \bmod 11 = (1 + 3^2) = 10$$

Indeks 10 er frit og der indsættes min alder:

Value	22	1	12			5	16			27	23
Index	0	1	2	3	4	5	6	7	8	9	10

Figure 11: Indsætter alder

Til sidst prøver vi at indsætte mit studienummer, som er 786842:

$$start(786842) = H(786842) \bmod 11 = 1$$

Da indeks 1 er optaget, må vi tjekke den næste:

$$h_i(786842) = (start(786842) + f(1)) \bmod 11 = (1 + 1^2) = 2$$

Indeks 2 er også optaget, så vi må inkrementere i med 1 og prøve igen:

$$h_i(786842) = (start(786842) + f(2)) \bmod 11 = (1 + 2^2) = 5$$

Indeks 5 er også optaget, så vi må inkrementere i med 1 og prøve igen:

$$h_i(786842) = (start(786842) + f(3)) \bmod 11 = (1 + 3^2) = 10$$

Indeks 10 er også optaget, så vi må inkrementere i med 1 og prøve igen:

$$h_i(786842) = (start(786842) + f(4)) \bmod 11 = 17 \bmod 11 = 6$$

Mellemresultatet er 17. Modulo indgår altid i formelen og giver derfor indeks 6:

$$17 \bmod 11 = 6$$

Indeks 6 er også optaget, så vi må inkrementere i med 1 og prøve igen:

$$h_i(786842) = (start(786842) + f(5)) \bmod 11 = (1 + 5^2) = 26 \bmod 11 = 4$$

Endelig fandt vi frem til indeks 4, som er tomt, og der placeres mit studienummer:

Value	22	1	12		786842	5	16			27	23
Index	0	1	2	3	4	5	6	7	8	9	10

Figure 12: Indsætter studienr